EmmaLee Peyton
CSCE 548

Pantry Tracker Deployment and System Test Document

1. Project Overview

Pantry Tracker is a multi-layer Java application designed to manage pantry inventory, ingredients, recipes, meals, and storage locations. The system uses a REST API built with Javalin, a MySQL database for persistent storage, and both a web frontend and console client for interacting with pantry data.

The application follows an n-tier architecture that separates the client layer, service layer, business logic layer, and data layer. This structure improves maintainability and allows different components of the system to evolve independently.

The system supports the following operations:

- Retrieve all records
- Retrieve a record by ID
- Insert new records
- Update existing records
- Delete records

Major components of the system include:

- MySQL database
- Java backend service using Javalin
- Business and DAO layers
- Web frontend written in HTML, JavaScript, and CSS
- Console client for testing API functionality

2. System Architecture

Client Layer: Users interact with the application through a web frontend and a Java console client.

Service Layer: The REST API is implemented using Javalin. This layer exposes endpoints that allow clients to retrieve, create, update, and delete data.

Business Layer: The business layer processes requests and applies application logic before passing data to the DAO layer.

Data Layer: DAO classes perform database operations and interact directly with the MySQL database.

Database: A MySQL database stores pantry information including ingredients, inventory items, recipes, and locations.

3. Technologies Used

- Java
- Javalin
- Jackson JSON library
- MySQL
- Maven
- HTML
- JavaScript
- CSS

4. Prerequisites

Before running the project, the following software must be installed:

- Java JDK (version 17 or newer)
- Maven
- MySQL Server
- A Java IDE such as IntelliJ IDEA or VS Code
- A modern web browser

5. Database Setup

1. Open MySQL Workbench or another MySQL client.

2. Run the database creation script: db_creation.sql

3. Run the sample data script: db_insertion.sql

4. Update database credentials in DBUtil.java so they match the local MySQL configuration.

6. Backend Setup and Execution

1. Navigate to the backend project folder: pantry_tracker

2. Build the project using Maven: mvn clean compile

3. Run the backend service: mvn exec:java

4. Verify the service is running by visiting http://localhost:7070/health

7. Frontend Setup

1. Navigate to the frontend folder.

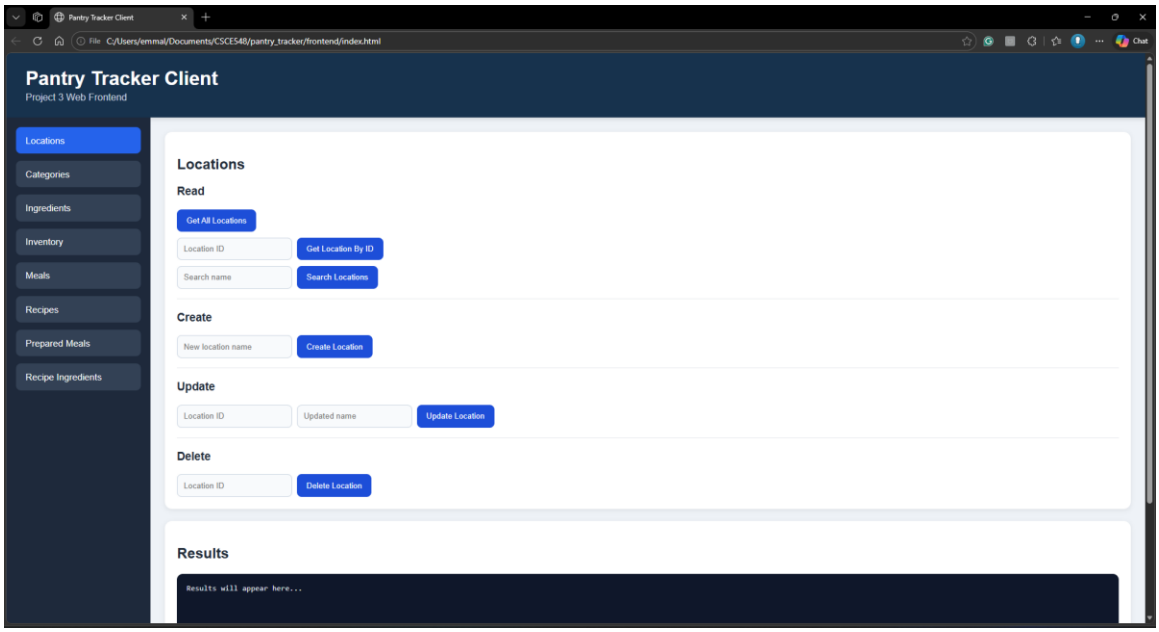
2. Open index.html in a web browser.

The frontend communicates with the backend REST API at http://localhost:7070.

8. System Testing

The system was tested by performing CRUD operations through the frontend interface and verifying database updates.

Starting Screen



Test 1 – Retrieve All Locations

Goal: Verify that the system can retrieve all storage locations.

Action: The frontend was used to request all location records.

Result: The system returned a list of all locations stored in the database.

Results


```
[
  {
    "id": 5,
    "name": "Basement Shelf"
  },
  {
    "id": 2,
    "name": "Freezer"
  },
  {
    "id": 1,
    "name": "Fridge"
  },
  {
    "id": 3,
    "name": "Pantry"
  },
  {
    "id": 4,
    "name": "Spice Rack"
  }
]
```


Test 2 – Create a New Location

Goal: Verify insertion of a new record.

Action: A new location was created through the frontend.

Result: The new record appeared in the database.

Locations

Read

Get All Locations

Location ID | Get Location By ID

Search name | Search Locations

Create

test shelf | Create Location

Update

Location ID | Updated name | Update Location

Delete

Location ID | Delete Location

Results

```
{
  "id": 0,
  "name": "test shelf"
}
```

Results

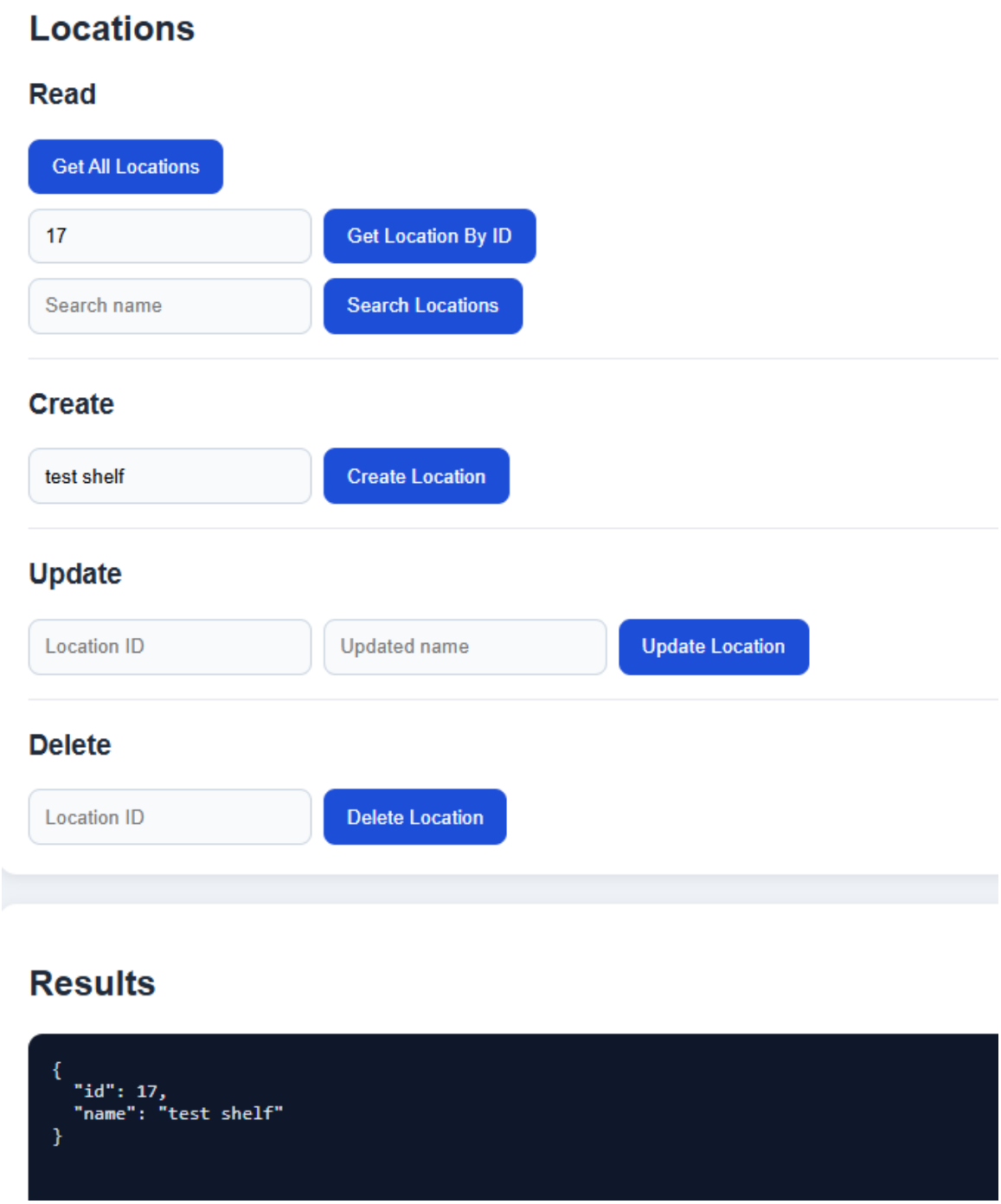

```
[
  {
    "id": 5,
    "name": "Basement Shelf"
  },
  {
    "id": 2,
    "name": "Freezer"
  },
  {
    "id": 1,
    "name": "Fridge"
  },
  {
    "id": 3,
    "name": "Pantry"
  },
  {
    "id": 4,
    "name": "Spice Rack"
  },
  {
    "id": 17,
    "name": "test shelf"
  }
]
```


Test 3 – Retrieve a Location by ID

Goal: Verify the API returns a single location.

Action: A request was made using a location ID.

Result: The correct location record was returned.

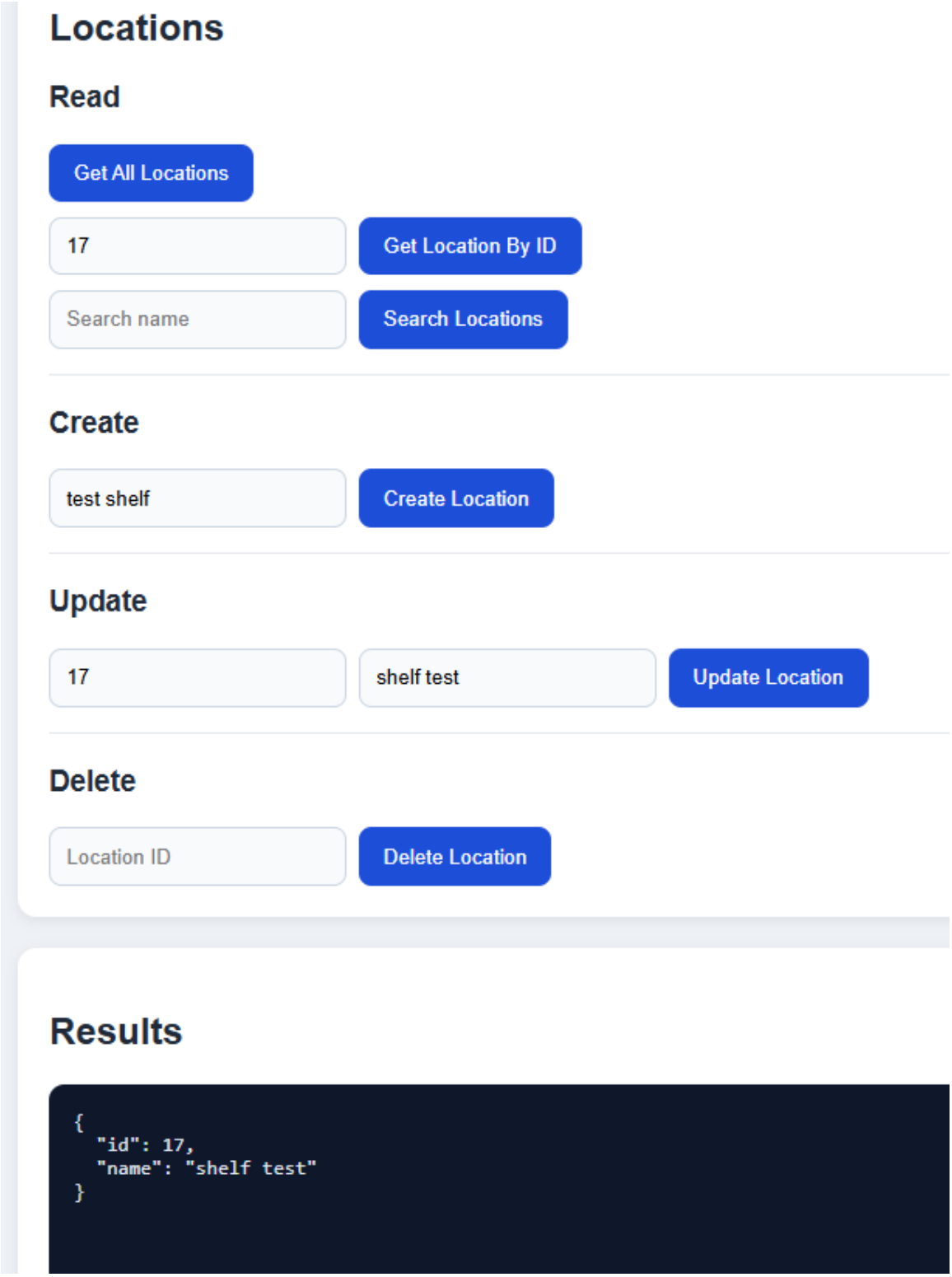


Test 4 – Update a Location

Goal: Verify existing records can be updated.

Action: A location record was modified.

Result: Updated values were saved successfully.

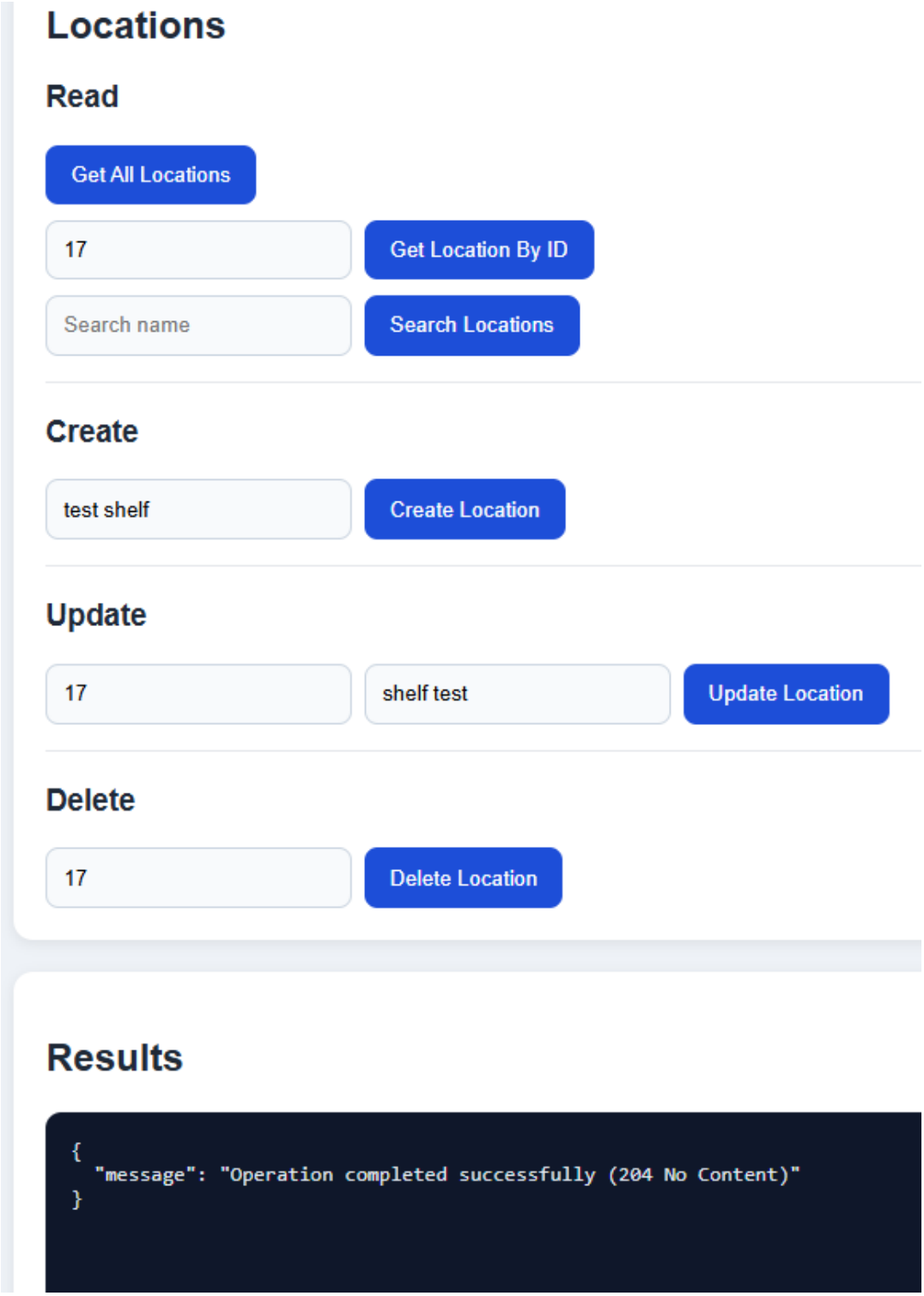


Test 5 – Delete a Location

Goal: Verify record deletion.

Action: A location record was deleted.

Result: The record was removed from the database.



Search name | Search Locations

Create

test shelf | Create Location

Update

17 | shelf test | Update Location

Delete

17 | Delete Location

Results


```
[
  {
    "id": 5,
    "name": "Basement Shelf"
  },
  {
    "id": 2,
    "name": "Freezer"
  },
  {
    "id": 1,
    "name": "Fridge"
  },
  {
    "id": 3,
    "name": "Pantry"
  },
  {
    "id": 4,
    "name": "Spice Rack"
  }
]
```

9. Graduate Credit

Prompts Used

The AI tool was used to generate structural code and assist with troubleshooting.

Example prompts included:

- Generate a Javalin REST API controller for pantry inventory items with CRUD endpoints.
- Create DAO classes for interacting with a MySQL database using JDBC.
- Generate a simple HTML and JavaScript frontend that communicates with a REST API.

Changes Made to Generated Output

- Adjusted field names to match the database schema
- Added validation in the business layer
- Updated endpoint URLs used by the frontend
- Added support for LocalDate JSON serialization
- Fixed dependency and import issues

Errors and Troubleshooting

- LocalDate serialization issue fixed using Jackson Java Time module
- Maven dependency refresh required for IDE recognition
- JSON request structure adjusted to match backend DTO objects

AI Tool Effectiveness

The AI tool was helpful for generating the initial structure of the application and speeding up development of repetitive components. However, manual adjustments were required to integrate generated code with the existing project architecture and database schema.

10. Conclusion

The Pantry Tracker system was successfully implemented using a layered architecture with a MySQL database, Java backend service, and web frontend.

All major CRUD operations were tested and confirmed to work correctly. The project demonstrates how an n-tier architecture can separate responsibilities between client, service, business, and data layers.